

МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное
бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по производственной практике

ПМ.02. Осуществление интеграции программных модулей

Студент

Кибардин Владимир Михайлович

Группа 23П-1

Специальность 09.02.07

Информационные системы и
программирование

Руководитель практики от колледжа:

Седов Алексей Сергеевич

Руководитель практики от организации:

_____ *Зязин Сергей Валерьевич*

подпись

УТВЕРЖДАЮ:

Начальник

Зязин Сергей Валерьевич

Наименование организации

Управление социального развития
администрации Слободского района

_____ / _____

подпись

расшифровка

М. П.

2025-2026 уч. год

ОГЛАВЛЕНИЕ

1. Характеристика объекта практики (юридический адрес, специализация, структура)
2. Описание рабочего места
3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение
4. Описание выполненных видов работ
 - 4.1. Проанализировать предметную область
 - 4.2. Спроектировать ERD, UML (Use case)
 - 4.3. Спроектировать на основе предметной области базу данных
 - 4.4. Произвести импорт данных в БД
 - 4.5. Разработать программный модуль на основе предметной области
 - 4.6. Произвести интеграцию с API
 - 4.7. Произвести отладку программных модулей
 - 4.8. Доработать программные модули обработкой исключительных ситуаций
 - 4.9. Рефакторинг программного кода
 - 4.10. Провести ручное тестирование
 - 4.11. Провести модульное тестирование
 - 4.12. Провести тестирование интеграции
 - 4.13. Фиксация результатов в системе контроля версий
 - 4.14. Разработка руководства оператора
 - 4.15. Разработка пояснительной записки
5. Заключение

1. Характеристика объекта практики (юридический адрес, специализация, структура)

Практика проходила в отделе социального развития администрации города Слободского (Кировская обл., г. Слободской, ул. Советская, 86). Деятельность организации связана с реализацией мер социальной поддержки, начислением пособий, ведением учёта льготных категорий граждан и поддержкой базы данных получателей услуг. В штатную структуру входят начальник отдела, несколько профильных специалистов по направлениям, бухгалтер и системный администратор. Всего в отделе трудится от 10 до 15 человек

2. Описание рабочего места

Практика проходила на моём личном ПК. На нём были установлены все необходимые для прохождения практики программы, включая офисный пакет, среды разработки, программы тестирования и прочие вспомогательные утилиты.

3. Состав программного и технического обеспечения, имеющегося на предприятии, их назначение

a. Основной ПК: Windows 11.

b. Учётная система: 1С:Предприятие.

c. Антивирус: 360 Total Security.

d. Техническое обеспечение: ПК с процессором Intel Xeon E5-2667 v2, 32 ГБ ОЗУ, SSD-накопитель

4. Описание выполненных видов работ

4.1 Анализ предметной области

Предметной областью является процесс аренды объектов недвижимости, принадлежащих муниципальному образованию. Система автоматизирует управление арендными отношениями между арендодателем (администрация) и арендаторами (физическими и юридическими лицами).

Основные бизнес-процессы включают:

- a. **Добавление объектов недвижимости** в реестр с указанием характеристик: адрес, кадастровый номер, тип, площадь, состояние;
- b. **Регистрацию арендаторов** (физические лица, ИП, юридические лица) с внесением данных: ИНН, ОГРН, контакты;
- c. **Заключение договоров аренды**, где фиксируются объект, арендатор, срок действия, ежемесячная плата и день платежа;
- d. **Внесение платежей** по договорам, отметка факта оплаты и учёт пеней при просрочке;
- e. **Контроль сроков аренды** — отслеживание истекающих договоров, автоматическое изменение статуса договора при завершении срока;
- f. **Учёт задолженности** — формирование списка должников на основе неоплаченных платежей;
- g. **Расторжение договоров** с освобождением объекта недвижимости и обновлением его статуса.

Выделены следующие категории пользователей:

- a. **Администратор** — полный доступ ко всем функциям системы;
- b. **Сотрудник отдела аренды** — работа с объектами, арендаторами, договорами, платежами;
- c. **Бухгалтер** — работа с договорами, платежами и должниками;
- d. **Арендатор** — просмотр своих договоров и платежей, ограниченный доступ к собственным данным.

На основе анализа построена диаграмма вариантов использования (Use Case), отображающая роли и их функциональные возможности (см. Рисунок 1).

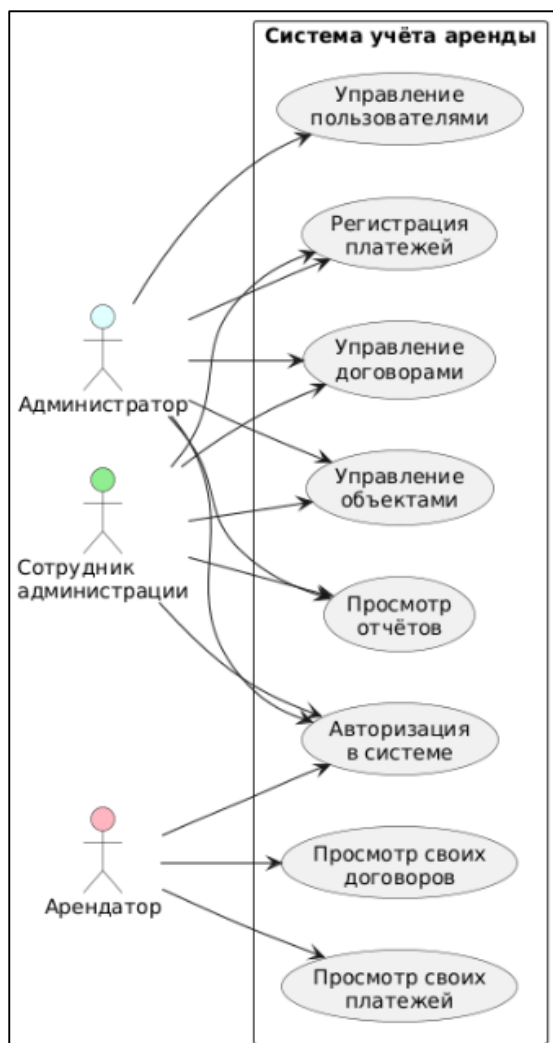


Рисунок 1 - Диаграмма вариантов использования

4.2 Проектирование ERD и UML (Use Case)

Диаграмма вариантов использования детализирует взаимодействие акторов с системой.

Арендатор регистрируется в системе, просматривает доступные объекты недвижимости, заключает договоры аренды и отслеживает свои платежи. Сотрудник отдела аренды управляет объектами и арендаторами, создает и расторгает договоры, контролирует сроки аренды. Бухгалтер ведет учёт поступлений, регистрирует платежи, формирует реестр должников и контролирует финансовую дисциплину арендаторов. Администратор управляет правами доступа пользователей, выполняет настройку системы и формирует сводные отчёты по арендной деятельности.

Логическая модель данных представлена в виде ER-диаграммы (см. Рисунок 2), которая отображает сущности:

- а. Пользователи (арендаторы, сотрудники, администратор)**
- б. Объекты недвижимости**
- с. Арендаторы**
- д. Договоры аренды**
- е. Платежи**

а также их взаимосвязи (один арендатор может иметь несколько договоров, один договор — несколько платежей и т.д.).

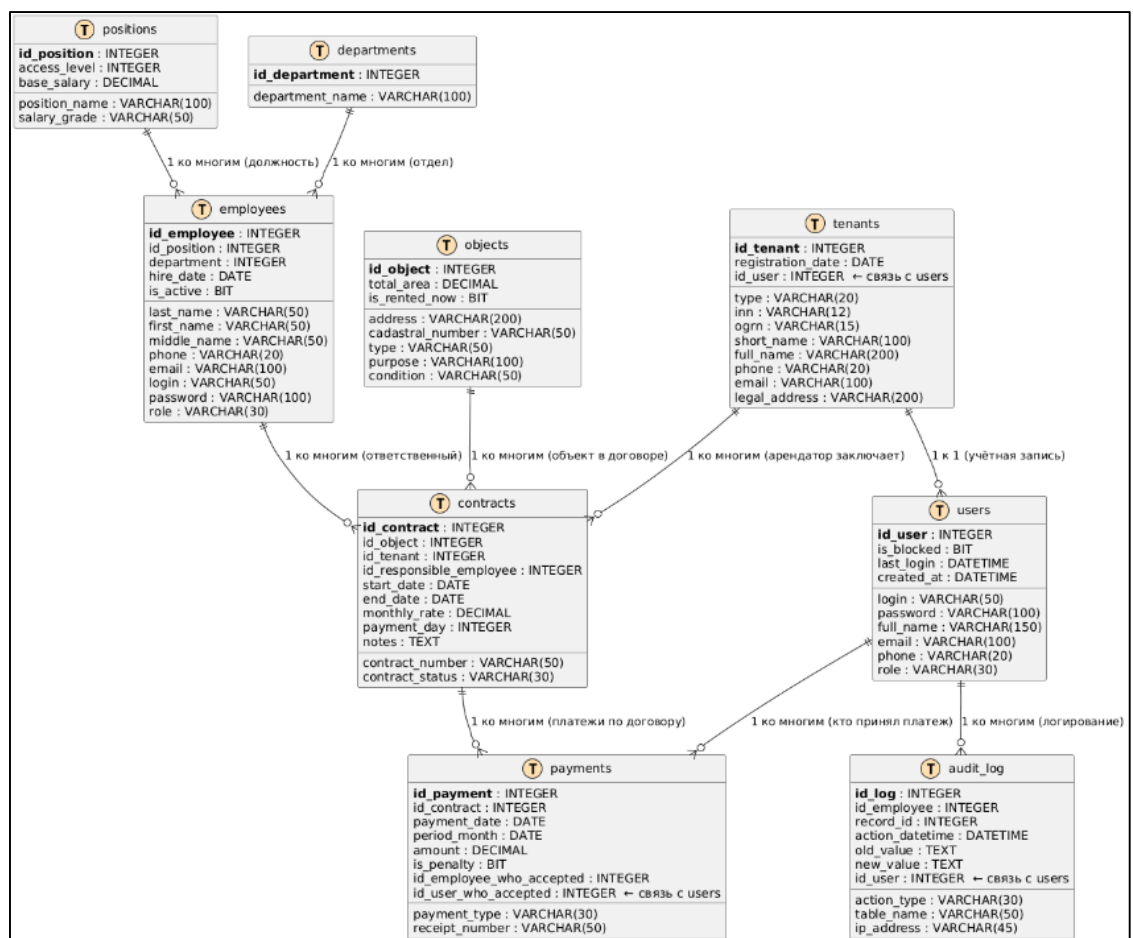


Рисунок 2 - ER-диаграмма базы данных

4.3 Проектирование базы данных

На основе ER-модели разработана реляционная база данных, нормализованная до третьей нормальной формы (3НФ). Определены следующие таблицы:

- a. **Users** – пользователи системы (логин, пароль, роль, ФИО, email, телефон, дата регистрации);
- b. **Tenants** – арендаторы (тип, ИНН, ОГРН, краткое и полное наименование, телефон, email, юридический адрес);
- c. **Objects** – объекты недвижимости (адрес, кадастровый номер, тип, площадь, назначение, состояние, признак аренды);
- d. **Contracts** – договоры аренды (номер, ID объекта, ID арендатора, ID ответственного сотрудника, дата начала и окончания, ежемесячная ставка, день оплаты, статус, примечания);
- e. **Payments** – платежи (ID договора, дата платежа, период, сумма, тип оплаты, признак пени, номер квитанции, ID сотрудника, принявшего оплату);
- f. **Employees** – сотрудники (ФИО, должность, телефон, email, логин, пароль, отдел);
- g. **Departments** – отделы (название отдела);
- h. **Positions** – должности (название должности, грейд, уровень доступа);
- i. **AuditLog** – журнал действий (ID пользователя, тип действия, таблица, ID записи, дата и время, старые и новые значения).

4.4 Импорт данных в БД

Для заполнения базы тестовыми данными разработан T-SQL скрипт, генерирующий более 1000 записей. Скрипт добавляет справочные данные (типы объектов, состояния, должности, отделы), 200 арендаторов (физические лица, ИП, юридические лица), 20 сотрудников (операторы, бухгалтеры, сотрудники отдела аренды), а также 300 договоров аренды со случайным распределением по объектам и арендаторам, с различными датами начала и окончания. Для каждого договора создаются платежи (от 1 до 12 записей на договор в зависимости от срока аренды), часть платежей помечена как пени для имитации просрочек.

Выполнение скрипта в среде SQL Server Management Studio продемонстрировано на Рисунке 3.



Рисунок 3 - Импорт тестовых данных

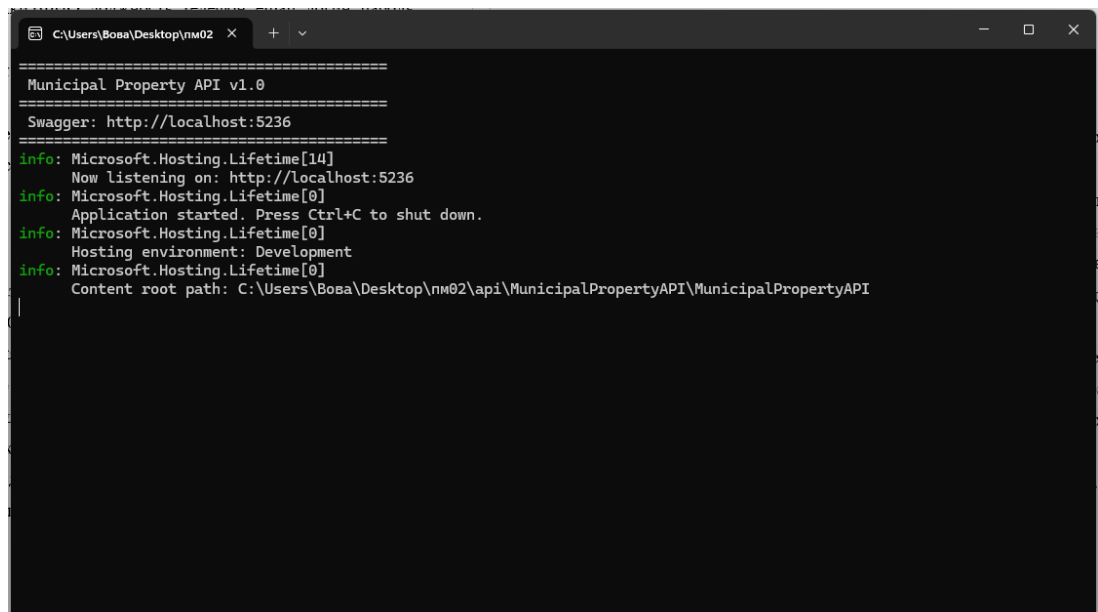
4.5 Разработка программного модуля

Программный комплекс состоит из двух частей: серверного Web API и клиентского настольного приложения (WPF).

Серверная часть реализована на ASP.NET Core 8. Предоставляет REST-эндпоинты для аутентификации, управления договорами, объектами недвижимости, арендаторами, платежами и справочными данными. Доступ к данным осуществляется через Entity Framework Core (подход Code First) с использованием SQL Server в качестве СУБД.

Аутентификация выполняется по паре логин/пароль, передаваемых в заголовках запросов (X-Login, X-Password). Ролевая модель разграничивает доступ: администратор, сотрудник отдела аренды, бухгалтер и арендатор имеют разные наборы доступных эндпоинтов.

Консоль запущенного сервера представлена на Рисунке 4.



```
=====  
Municipal Property API v1.0  
=====  
Swagger: http://localhost:5236  
=====  
info: Microsoft.Hosting.Lifetime[14]  
Now listening on: http://localhost:5236  
info: Microsoft.Hosting.Lifetime[0]  
Application started. Press Ctrl+C to shut down.  
info: Microsoft.Hosting.Lifetime[0]  
Hosting environment: Development  
info: Microsoft.Hosting.Lifetime[0]  
Content root path: C:\Users\Boba\Desktop\nm02\api\MunicipalPropertyAPI\MunicipalPropertyAPI
```

Рисунок 4 - Командная строка API ASP.NET

Клиентская часть представляет собой настольное приложение на Windows Presentation Foundation (WPF) с архитектурой, построенной на принципах MVVM (Model-View-ViewModel). Пользовательский интерфейс (показан на Рисунке 5) адаптируется под роль авторизованного пользователя: администратору доступны все разделы, сотруднику отдела аренды — управление объектами и договорами, бухгалтеру — платежи и должники, арендатору — только собственные договоры и платежи.

Взаимодействие с сервером инкапсулировано в сервисе ApiClient, который выполняет HTTP-запросы к REST-эндпоинтам, передаёт учётные данные в заголовках (X-Login, X-Password) и десериализует JSON-ответы. Для сериализации используется библиотека Newtonsoft.Json.

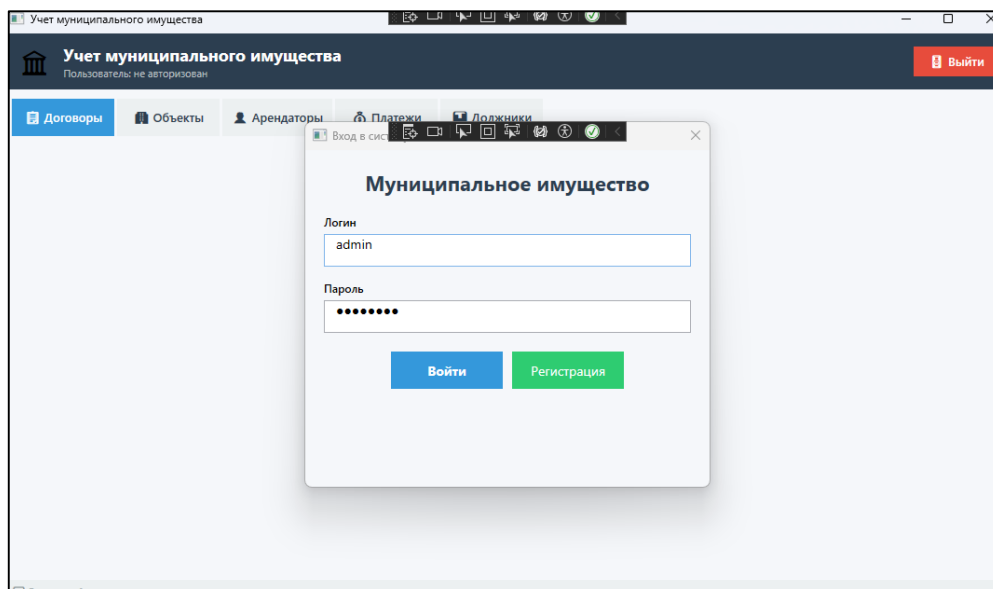


Рисунок 5 - Пользовательский интерфейс

4.6 Интеграция с API

Интеграция между клиентским приложением и сервером выполнена путём вытеснения прямых SQL-запросов из кода клиента в пользу REST-API вызовов. Вся логика взаимодействия с сервером вынесена в сервис *ApiClient*, который централизованно формирует HTTP-запросы, добавляет обязательные заголовки аутентификации (X-Login, X-Password) и обрабатывает полученные ответы.

При возникновении ошибок (отсутствие сети, недоступность сервера, код ответа 4xx/5xx) сервис генерирует исключения, которые перехватываются на уровне представлений и выводят пользователю понятные сообщения в виде информационных диалогов.

Пример интеграции — окно добавления/редактирования договора (Рисунок 6), где список доступных объектов недвижимости и арендаторов загружается через *Get* — запросы к эндпоинтам `/api/propertyobjects` и `/api/tenants`, а сохранение договора выполняется через *POST*-запрос к `/api/contracts`.

ID	Номер	Объект	Арендатор	Начало	Окончание	Ставка	Статус
151	Д-20260809-145217	г. Москва	Boris Borisov	09.08.2026	09.08.2027	500,000.00	действует
144	CONTRACT-144	г. Слободской, ул. Пушкина, д. 145	ООО Kazbek	24.05.2026	24.05.2027	14,000.00	active
96	CONTRACT-96	г. Слободской, ул. Северная, д. 97	Aleksandr Alekseev	24.05.2026	24.05.2027	26,000.00	active
120	CONTRACT-120	г. Слободской, ул. Ленина, д. 121	IP Sidorov Sidor	24.05.2026	24.05.2027	10,000.00	active
24	CONTRACT-24	г. Слободской, ул. Пушкина, д. 25	IP Rakov Roman	24.05.2026	24.05.2027	14,000.00	active
48	CONTRACT-48	г. Слободской, ул. Садовая, д. 49	ООО Rocky Mountains	24.05.2026	24.05.2027	18,000.00	active
72	CONTRACT-72	г. Слободской, ул. Молодёжная, д. 73	Shcherbak Shcherbakov	24.05.2026	24.05.2027	22,000.00	active
73	CONTRACT-73	г. Слободской, ул. Строителей, д. 74	Yury Yuriev	24.04.2026	24.04.2027	23,000.00	active
49	CONTRACT-49	г. Слободской, ул. Набережная, д. 50	IP Andesov Andrey	24.04.2026	24.04.2027	19,000.00	active
25	CONTRACT-25	г. Слободской, ул. Гоголя, д. 26	ООО Lev	24.04.2026	24.04.2027	15,000.00	active
1	CONTRACT-1	г. Слободской, ул. Кирова, д. 2	IP Ivanov Ivan	24.04.2026	24.04.2027	11,000.00	active
121	CONTRACT-121	г. Слободской, ул. Кирова, д. 122	ООО Telets	24.04.2026	24.04.2027	11,000.00	active
97	CONTRACT-97	г. Слободской, ул. Южная, д. 98	Galina Grigorieva	24.04.2026	24.04.2027	27,000.00	active
145	CONTRACT-145	г. Слободской, ул. Гоголя, д. 146	IP Everestov Eduard	24.04.2026	24.04.2027	15,000.00	active
146	CONTRACT-146	г. Слободской, ул. Мира, д. 147	ООО Alpy	24.03.2026	24.03.2027	16,000.00	active
98	CONTRACT-98	г. Слободской, ул. Вятская, д. 99	Zahar Zaharov	24.03.2026	24.03.2027	28,000.00	active
122	CONTRACT-122	г. Слободской, ул. Свердлова, д. 123	ООО Bliznetsy	24.03.2026	24.03.2027	12,000.00	active
2	CONTRACT-2	г. Слободской, ул. Свердлова, д. 3	ООО Rassvet	24.03.2026	24.03.2027	12,000.00	active
26	CONTRACT-26	г. Слободской, ул. Мина, д. 27	ООО Deva	24.03.2026	24.03.2027	16,000.00	active

Рисунок 6 - Отображение данных, полученных через API

4.7 Отладка программных модулей

Отладка выполнялась на всех этапах разработки с использованием встроенных средств Visual Studio (точки останова, пошаговое выполнение, просмотр значений переменных) и Postman для тестирования REST-эндпоинтов.

В процессе отладки серверной части проверялись:

- корректность маршрутизации запросов;
- валидация входных данных;
- правильность формирования HTTP-ответов для различных сценариев (успешное выполнение, ошибки валидации, отсутствие прав доступа).

В клиентской части основное внимание уделялось:

- правильности формирования HTTP-запросов с заголовками аутентификации;
- обработке ответов сервера (успешных и ошибочных);
- корректному отображению данных в интерфейсе.

В ходе отладки были выявлены и устранены следующие проблемы:

- циклические ссылки при сериализации JSON (исправлены с помощью проекции данных на анонимные типы и DTO);

- b. ошибки при передаче NULL-значений в поля, не допускающие NULL (исправлены добавлением проверок и обработки NULL);
- c. некорректная обработка 401 Unauthorized на клиенте (добавлена обработка и скрывание ошибок для ролей без доступа).

4.8 Доработка обработкой исключительных ситуаций

В процессе разработки и отладки программных модулей была выполнена доработка системы с целью обработки исключительных ситуаций на всех уровнях приложения. Обработка реализована как на серверной, так и на клиентской стороне.

На серверной части (ASP.NET Core 8):

- a. Добавлены блоки try-catch в методы контроллеров для перехвата ошибок при работе с базой данных и бизнес-логикой.
- b. Настроены проверки на существование запрашиваемых сущностей (объекты, арендаторы, договоры) с возвратом 404 Not Found.
- c. Добавлены валидации входных данных: проверка обязательных полей, форматов (ИНН, кадастровый номер), диапазонов дат.
- d. Для ошибок аутентификации и авторизации возвращаются статусы 401 Unauthorized и 403 Forbidden с соответствующими сообщениями.

На клиентской части (WPF):

- a. Каждый асинхронный метод, выполняющий HTTP-запрос, обернут в блок try-catch.
- b. Добавлена специальная обработка ошибки 401 Unauthorized — для ролей, не имеющих доступа к определённым разделам, ошибка логируется без отображения пользователю, а интерфейс корректно очищается.
- c. Предусмотрена обработка сетевых ошибок: при недоступности сервера пользователь получает соответствующее уведомление с предложением проверить подключение.

- d. Реализована валидация пользовательского ввода на уровне представлений (проверка заполнения полей, корректности числовых значений, форматов дат) с выводом предупреждений до отправки запроса на сервер.

В результате доработки система стала устойчивой к ошибкам: некорректные действия пользователей не приводят к сбоям, а ошибки сервера обрабатываются без потери данных и с понятным для пользователя оповещением.

4.9 Рефакторинг программного кода

На завершающем этапе разработки был проведён рефакторинг программного кода с целью повышения его читаемости, поддерживаемости и производительности. Рефакторинг выполнялся как для серверной, так и для клиентской частей приложения.

На серверной части (ASP.NET Core 8):

- a. Выделение общих методов в базовый контроллер – логика авторизации и получения текущего пользователя вынесена в абстрактный класс `BaseApiController`, что позволило сократить дублирование кода во всех контроллерах.
- b. Использование DTO вместо прямого возврата сущностей – для всех эндпоинтов внедрены DTO-объекты, что предотвращает утечку внутренней структуры базы данных и циклические ссылки при сериализации.
- c. Переименование методов и переменных – приведение имён к единому стилю (C# Coding Conventions), улучшение читаемости кода.
- d. Удаление неиспользуемых `using`-директив и мёртвого кода – очистка проекта от устаревших и закомментированных фрагментов.

На клиентской части (WPF):

- a. Инкапсуляция HTTP-логики в отдельный сервис – все запросы к API вынесены в класс `ApiClient`, что централизует управление авторизацией, обработку ошибок и настройку заголовков.

- b. Унификация обработки ошибок – создан единый подход к перехвату исключений в асинхронных методах и отображению сообщений пользователю.
- c. Выделение общих методов в базовый класс – для представлений (Views) реализованы общие методы загрузки данных (LoadData) и обработки поиска с задержкой (DispatcherTimer), что исключило дублирование кода в каждом пользовательском контроле.
- d. Улучшение читаемости XAML-разметки –**格式化** разметки, выравнивание элементов, использование единых стилей для кнопок и таблиц.
- e. Переименование элементов управления – присвоение осмысленных имён (txtSearch, dgContracts, btnRefresh) для улучшения понимания кода.

4.10 Ручное тестирование

Ручное тестирование проводилось по заранее составленным тест-кейсам, охватывающим функциональность всех ролей: администратор, сотрудник отдела аренды, бухгалтер, арендатор. Всего разработано и выполнено **16 тестов** (10 тест-кейсов для клиентского приложения и 6 запросов в Postman для тестирования API). В таблице 1 приведён фрагмент тест-кейсов (полный перечень – в репозитории).

ТЕСТ-КЕЙС №2: Вход с некорректными данными:

Поле	Значение
ID	ТС-02
Название теста	Вход с некорректными данными
Приоритет	Высокий
Предусловия	Запущено приложение
Шаги воспроизведения	1. В поле «Логин» ввести wto64ng43 2. В поле «Пароль» ввести wt64ong33 3. Нажать «Войти»
Ожидаемый результат	Сообщение об ошибке «Неверный логин или пароль»

Фактический результат	Соответствует
Статус	Пройден

Все тесты завершились успешно, что подтверждает соответствие реализованной функциональности требованиям и готовность системы к эксплуатации.

4.11 Модульное тестирование

Модульное тестирование проведено для клиентского сервиса ApiClient с использованием фреймворков **NUnit** и **Moq**. Были протестированы сценарии успешного выполнения запросов, ошибок аутентификации, сетевых исключений и проверки добавления заголовков. Результаты тестирования показаны на Рисунке 7.

Листинг 1 демонстрирует пример теста.

```
[Test]
public void IsAuthorized_ShouldReturnTrue_WhenCredentialsSet()
{
    _apiClient.SetCredentials("volkov_andrey", "admin123");
    var result = _apiClient.IsAuthorized();
    Assert.That(result, Is.True);
}
```

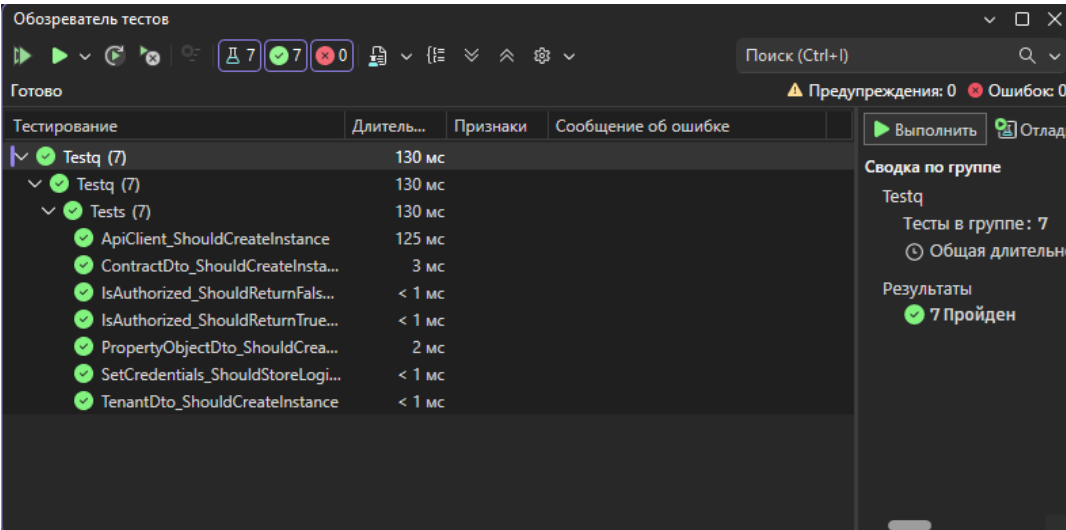


Рисунок 7 - Результат модульного тестирования

4.12 Интеграционное тестирование

Интеграционные тесты выполнены для API с помощью WebApplicationFactory и In-Memory базы данных. Сценарии включают:

проверку доступности эндпоинтов без авторизации (ожидаемый статус 401 Unauthorized);

- доступ к данным с правами администратора
- разграничение прав доступа для ролей: арендатор не может просматривать объекты и арендаторов;
- доступ бухгалтера к сводке платежей и списку должников.

Все тесты прошли успешно (Рисунок 8).

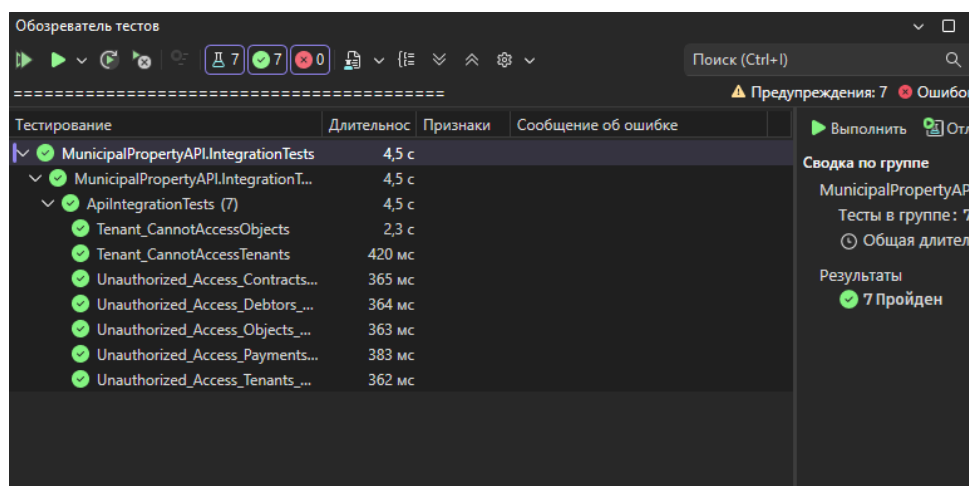


Рисунок 8 - Результаты интеграционного тестирования

4.13 Фиксация результатов в системе контроля версий

Исходный код проекта размещён в системе контроля версий **Git**. В процессе работы регулярно выполнялись коммиты с осмысленными сообщениями, фиксирующие ключевые изменения: создание структуры проекта, реализацию API, перевод клиента на API, исправление ошибок, добавление тестов и документации. Репозиторий обеспечивает сохранность истории разработки и возможность отката к предыдущим состояниям.

Репозиторий доступен публично:

<https://github.com/vova63428/PraktikaProizvPM02>

4.14 Разработка руководства оператора

Разработан документ «Руководство оператора», описывающий порядок работы с приложением для всех категорий пользователей. Документ содержит инструкции по запуску, входу в систему, основным операциям (управление объектами, арендаторами, договорами, платежами) и действиям в нештатных ситуациях. Основное содержание руководства приведено в репозитории.

4.15 Разработка пояснительной записки

Пояснительная записка содержит описание архитектуры системы, перечень реализованных функций, обоснование технических решений, результаты тестирования и инструкции по развёртыванию. Документ позволяет получить полное представление о программном комплексе, его структуре и логике работы. Полный текст записки представлен в репозитории.

5 Заключение

В ходе выполнения практики по модулю «Осуществление интеграции программных модулей» была достигнута поставленная цель – разработана информационная система «Управление арендой муниципального имущества», состоящая из серверного API и клиентского приложения.

Решены следующие задачи:

Проведён анализ предметной области, построены UML-диаграммы (Use Case) и ER-модель.

Спроектирована нормализованная база данных и выполнено её развёртывание с заполнением тестовыми данными.

Разработан программный модуль в виде Web API на ASP.NET Core и WPF-клиента.

Реализована интеграция клиента с API через HTTP-запросы с авторизацией.

Выполнено тестирование (модульное, интеграционное, ручное) и подготовлена документация.

Разработанная система позволяет автоматизировать учёт арендных отношений, контролировать сроки договоров и платежи, формировать отчётность и повысить прозрачность работы с муниципальным имуществом.